香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

# Graph Algorithms (I)

- Basic Definitions and Applications
- Graph Connectivity and Graph Traversal
- Connectivity in Directed Graphs
- DAGs & Topological Ordering

Yanlin Zhang & Shangqi Lu | DSAA 2043 Fall 2026

# Basic Definitions and Applications

# Definition



- A graph: a group of vertices and edges that are used to connect these vertices

- Definition: A graph G can be defined as an ordered set $G(V, E)$
  - $V$ represents the set of vertices/nodes
  - $E$ represents the set of edges which are used to connect $V$

# Applications: Social Network

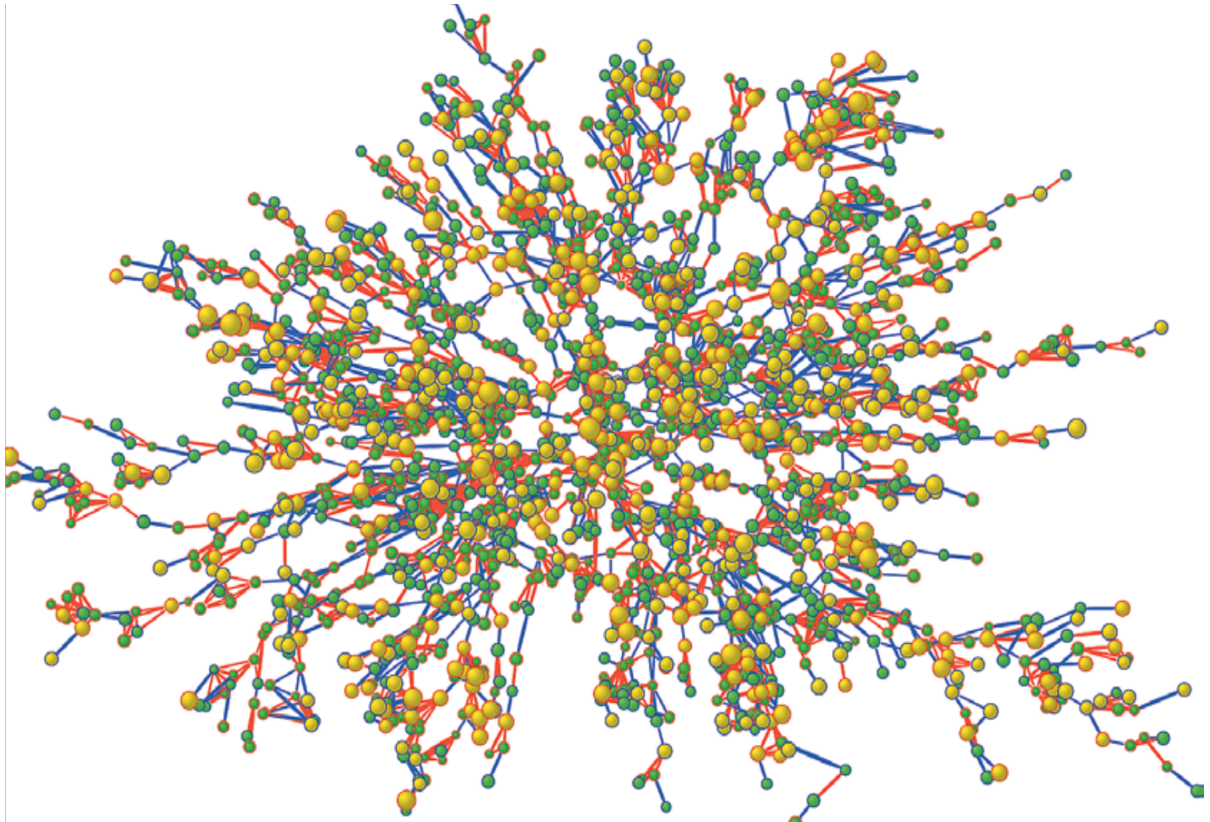
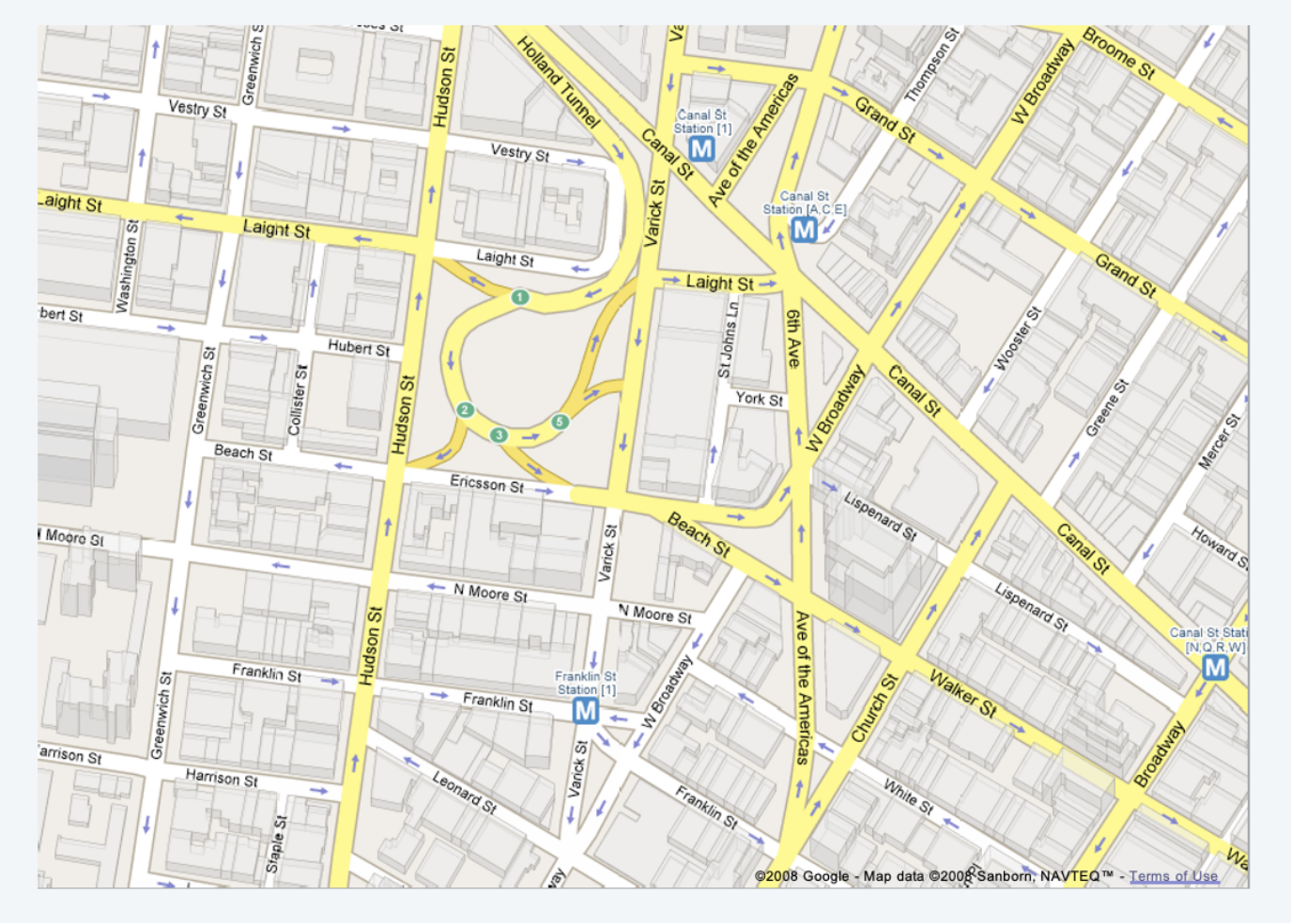


**Figure 1. Largest Connected Subcomponent of the Social Network in the Framingham Heart Study in the Year 2000.**

Each circle (node) represents one person in the data set. There are 2200 persons in this subcomponent of the social network. Circles with red borders denote women, and circles with blue borders denote men. The size of each circle is proportional to the person's body-mass index. The interior color of the circles indicates the person's obesity status: yellow denotes an obese person (body-mass index, ≥30) and green denotes a nonobese person. The colors of the ties between the nodes indicate the relationship between them: purple denotes a friendship or marital tie and orange denotes a familial tie.

# Road Network

Node = intersection; edge = street.



# More Applications

| graph | node | edge |
|---|---|---|
| **communication** | telephone, computer | fiber optic cable |
| **circuit** | gate, register, processor | wire |
| **mechanical** | joint | rod, beam, spring |
| **financial** | stock, currency | transactions |
| **transportation** | street intersection, airport | highway, airway route |
| **internet** | class C network | connection |
| **game** | board position | legal move |
| **social relationship** | person, actor | friendship, movie cast |
| **neural network** | neuron | synapse |
| **protein network** | protein | protein-protein interaction |
| **molecule** | atom | bond |

# Sequential Representation

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

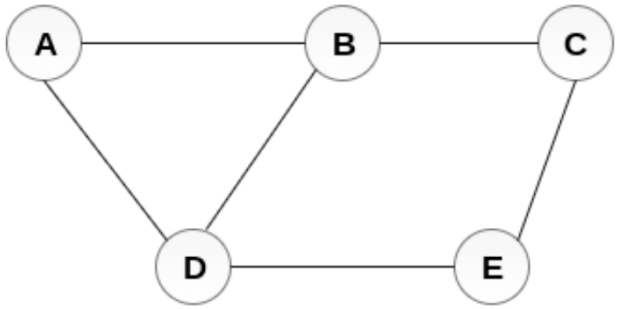
- Use adjacency matrix to store the mapping represented by vertices and edges

- In adjacency matrix, the rows and columns are represented by the graph vertices

- For a graph having $n$ vertices, the adjacency matrix will have a dimension $n \times n$

# Sequential Representation

- Undirected: an entry $A_{ij}$ in the adjacency matrix will be 1 if there exists an edge between $v_i$ and $v_j$.




Undirected Graph

| | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 1 | 0 |
| B | 1 | 0 | 1 | 1 | 0 |
| C | 0 | 1 | 0 | 0 | 1 |
| D | 1 | 1 | 0 | 0 | 1 |
| E | 0 | 0 | 1 | 1 | 0 |

Adjacency Matrix

# Linked Representation



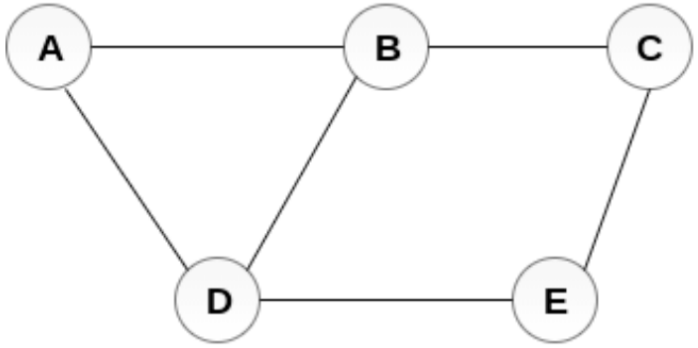
- An adjacency list is used to store the Graph into the computer's memory

- An adjacency list is maintained for each node present in the graph which stores the node value and a pointer to the next adjacent node to the respective node
  - A linked list to store each node's neighbors

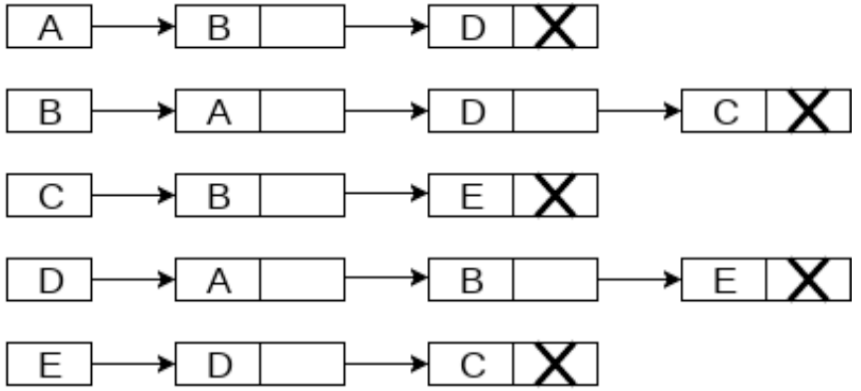
- If all the adjacent nodes are traversed, then store the NULL in the pointer field of last node of the list

# Linked Representation

- Undirected: The sum of the lengths of adjacency lists is equal to the twice of the number of edges




Undirected Graph




Adjacency List

# Terminology



- Path: a sequence of edges connecting initial node $v_0$ to terminal node $v_n$
- Closed Path: A path where the initial node is same as terminal node, i.e., $v_0 = v_n$
- Simple Path: all the nodes of the path are distinct, with the exception $v_0 = v_n$
- Closed Simple Path: a simple path with $v_0 = v_n$
- Cycle: a path which has no repeated edges or vertices except the first and last vertices
- Adjacent Nodes: two nodes $u$ and $v$ are connected via an edge $e$
  - the nodes $u$ and $v$ are also called as neighbors
- Degree of a Node: the number of edges (neighbors) that are connected with the node
  - A node with degree 0 is called as isolated node

# Terminology



- Connected Graph: a graph in which a path exists between every two vertices $u$ and $v$ in $V$
  - There are no isolated nodes in connected graph
- Complete Graph: a graph in which there is an edge between each pair of vertices
  - A complete graph contain $n(n-1)/2$ edges where $n$ is the number of nodes in the graph
- Weighted Graph: each edge is assigned with some data such as length or weight
  - The weight of an edge $e$, $w(e)$
- Digraph: each edge of the graph is associated with some direction
  - The traversing can be done only in the specified direction

# Graph Traversal

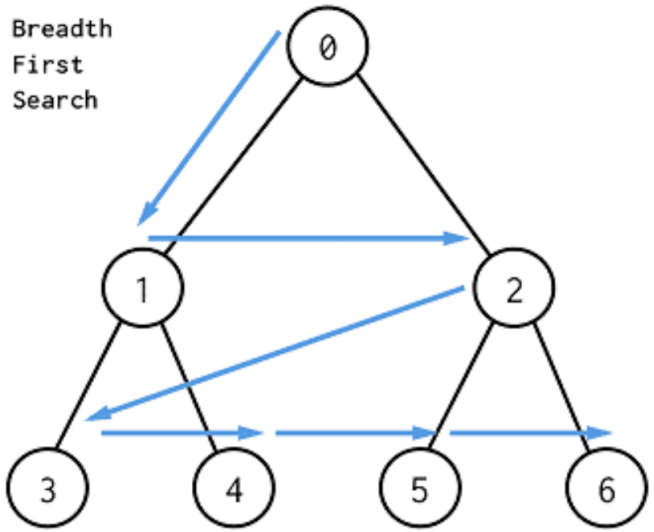
# Connectivity

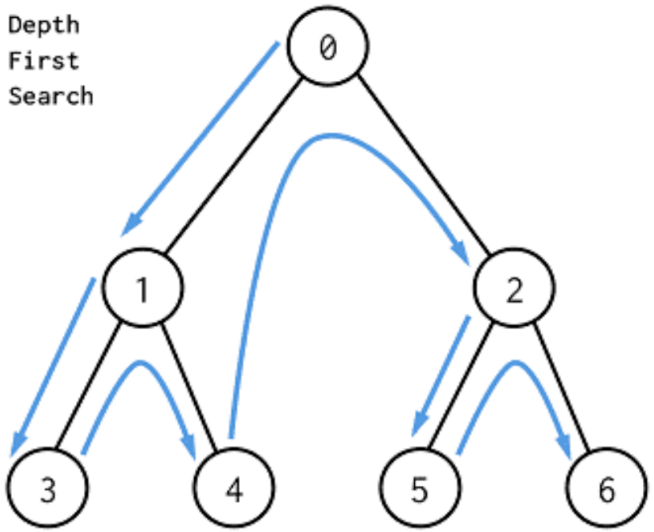


- $s-t$ connectivity problem: Given two nodes $s$ and $t$, is there a path between $s$ and $t$?
- $s-t$ shortest path problem: Given two nodes $s$ and $t$, what is the length of a shortest path between $s$ and $t$?
- Applications
  - Friendster
  - Maze traversal
  - Kevin Bacon number
  - Fewest hops in a communication network

# Graph Traversal

- Traversing the graph means examining all the nodes and vertices of the graph
- Two standard methods to traverse graphs
  - Breadth First Search
  - Depth First Search



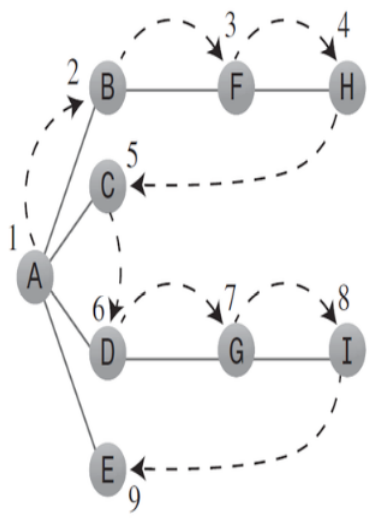



# Depth First Search (Stack-Based)

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

- DFS: starts with the initial node, and then goes to deeper and deeper until we find the goal node or the node which has no children. The algorithm, then backtracks from the dead end towards the most recent node that is yet to be completely unexplored.
  - Step 1: SET STATUS as UNVISITED (ready state) for each node in G
  - Step 2: Push the starting node A on the stack
  - Step 3: Repeat Steps 4 and 5 until STACK is empty
  - Step 4: Pop the top node N. If node N is VISITED, repeat Step 4; Otherwise, process it and set its STATUS as VISITED (processed state)
  - Step 5: Push on the stack all the neighbours of N with STATUS UNVISITED
  - Step 6: EXIT

# Depth First Search (Stack-Based)




| Event | Stack |
|---|---|
| | A |
| Visit & Pop A | |
| Push A's unvisited neighbours | EDCB |
| Visit & Pop B | EDC |
| Push B's unvisited neighbours | EDCF |
| Visit & Pop F | EDC |
| Push F's unvisited neighbours | EDCH |
| Visit & Pop H | EDC |
| Push H's unvisited neighbours | EDC |
| Visit & Pop C | ED |
| Push C's unvisited neighbours | ED |
| Visit & Pop D | E |
| Push D's unvisited neighbours | EG |
| Visit & Pop G | E |
| Push G's unvisited neighbours | EI |
| Visit & Pop I | E |
| Push I's unvisited neighbours | E |
| Visit & Pop E | |
| Done | |

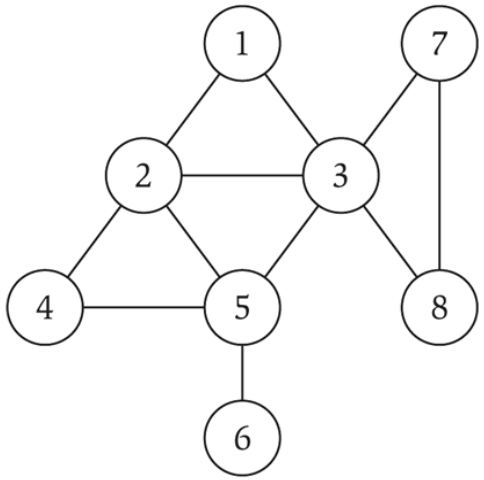
- Rule 1: Push every unvisited neighbour (if there is one) of the current vertex on the stack
- Rule 2: If you can't carry out Rule 1 because there are no more unvisited vertices, pop vertices from the stack (if possible) until a vertex is unvisited, mark it visited, and make it the current vertex
- Rule 3: If you can't carry out Rule 1&2 because the stack is empty, you're done

**Time complexity** $O(m+n)$, with an adjacency list

# Depth First Search (Recursion-Based)



```
DFS-recursive(G, s):
    mark s as visited
    for all neighbours w of s in Graph G:
        if w is not visited:
            DFS-recursive(G, w)
```



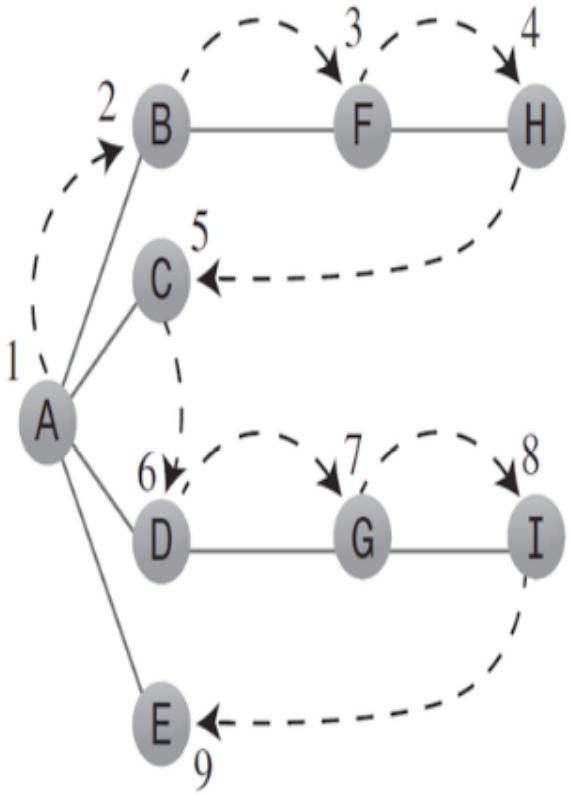

Assume that we follow neighbours with smaller ID first

DFS-recursive$(G, 1)$ = $[1, 2, 4, 5, 6, \mathbf{3}, 7, 8]$

**Time complexity** $O(n + m)$, when implemented using an adjacency list.

# Traversals

Try to implement the two DFS traversal algorithms (lab)

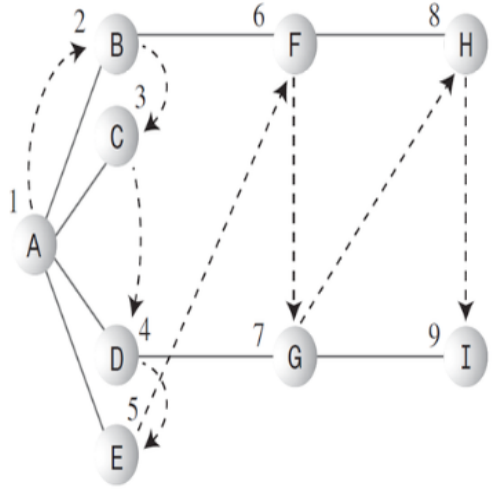


# Breadth First Search



- BFS: starts traversing the graph from root node and explores all the neighbours. Then, it selects the nearest node and explore all the unexplored nodes. It follows the same process for each of the nearest node until it finds the goal.
  - Step 1: SET STATUS = 1 (ready state) for each node in G
  - Step 2: Enqueue the starting node A and set its STATUS = 2 (waiting state)
  - Step 3: Repeat Steps 4 and 5 until QUEUE is empty
  - Step 4: Dequeue a node N. Process it and set its STATUS = 3 (processed state).
  - Step 5: Enqueue all neighbours of N in the ready state (STATUS = 1) and set their STATUS = 2
  - Step 6: EXIT

# Breadth First Search




| Event | Queue (Front to Rear) |
|---|---|
| Visit A | |
| Visit B | B |
| Visit C | BC |
| Visit D | BCD |
| Visit E | BCDE |
| Remove B | CDE |
| Visit F | CDEF |
| Remove C | DEF |
| Remove D | EF |
| Visit G | EFG |
| Remove E | FG |
| Remove F | G |
| Visit H | GH |
| Remove G | H |
| Visit I | HI |
| Remove H | I |
| Remove I | |
| Done | |

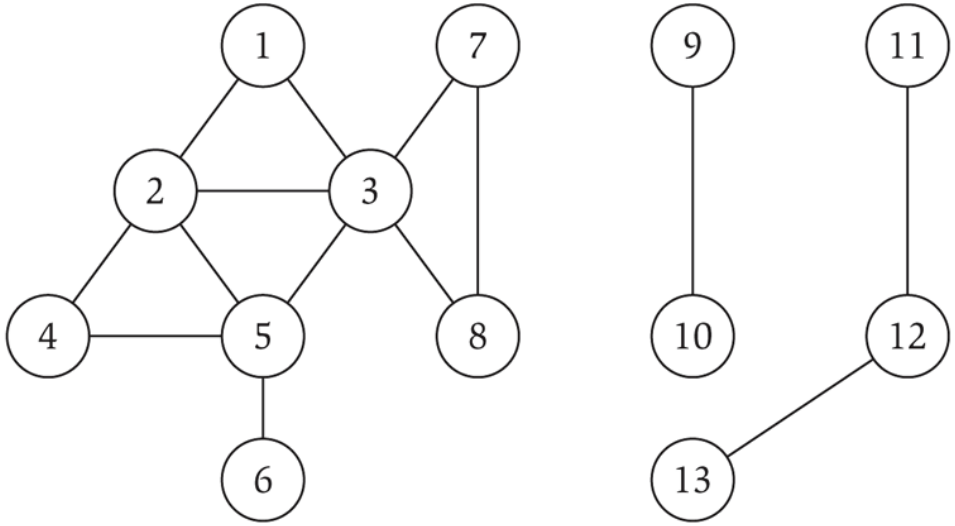
- Rule 1: Visit the next unvisited vertex (if there is one) that's adjacent to the current vertex, and insert it into the queue
- Rule 2: If you can't carry out Rule 1 because there are no more unvisited vertices, remove a vertex from the queue (if possible) and make it the current vertex
- Rule 3: If you can't carry out Rule 1&2 because the queue is empty, you're done

**Time complexity** $O(m+n)$, when implemented using an adjacency list

# Graph Connectivity

# Connected Component

- Connected component: find all nodes reachable from $s$




Connected component containing node 1 is [1, 2, 3, 4, 5, 6, 7, 8 ]

# Connected Component

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

Connected component: find all nodes reachable from $s$

---

$R$ will consist of nodes to which $s$ has a path

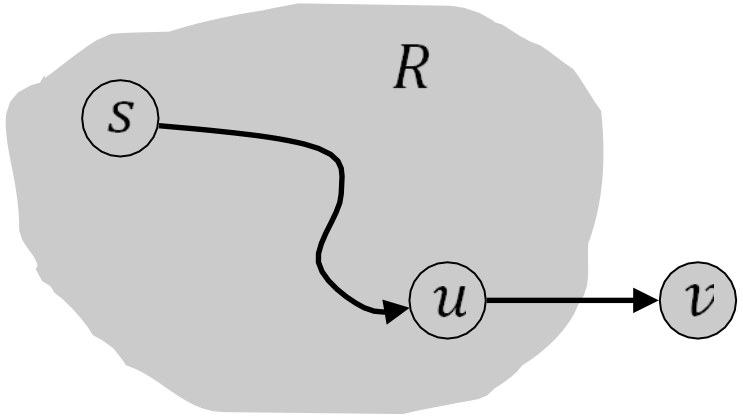
Initially $R = \{s\}$

While there is an edge $(u, v)$ where $u \in R$ and $v \notin R$

  Add $v$ to $R$

Endwhile

---




it's safe to add $v$

Theorem. Upon termination, $R$ is the connected component containing $s$

- BFS = explore in order of distance from $s$
- DFS = explore in a different way

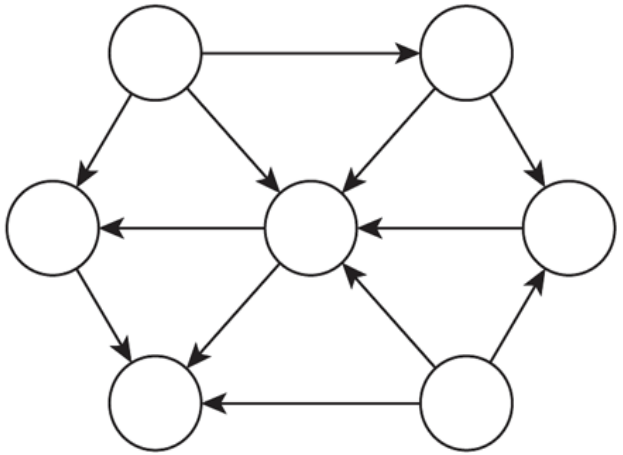
# Connectivity in Directed Graphs

# Directed Graph



Notation: $G = (V, E)$

- Edge $(u, v)$ leaves node $u$ and enters node $v$



Ex. Web graph: hyperlink points from one web page to another

Orientation of edges is crucial

Modern web search engines exploit hyperlink structure to rank web pages by importance

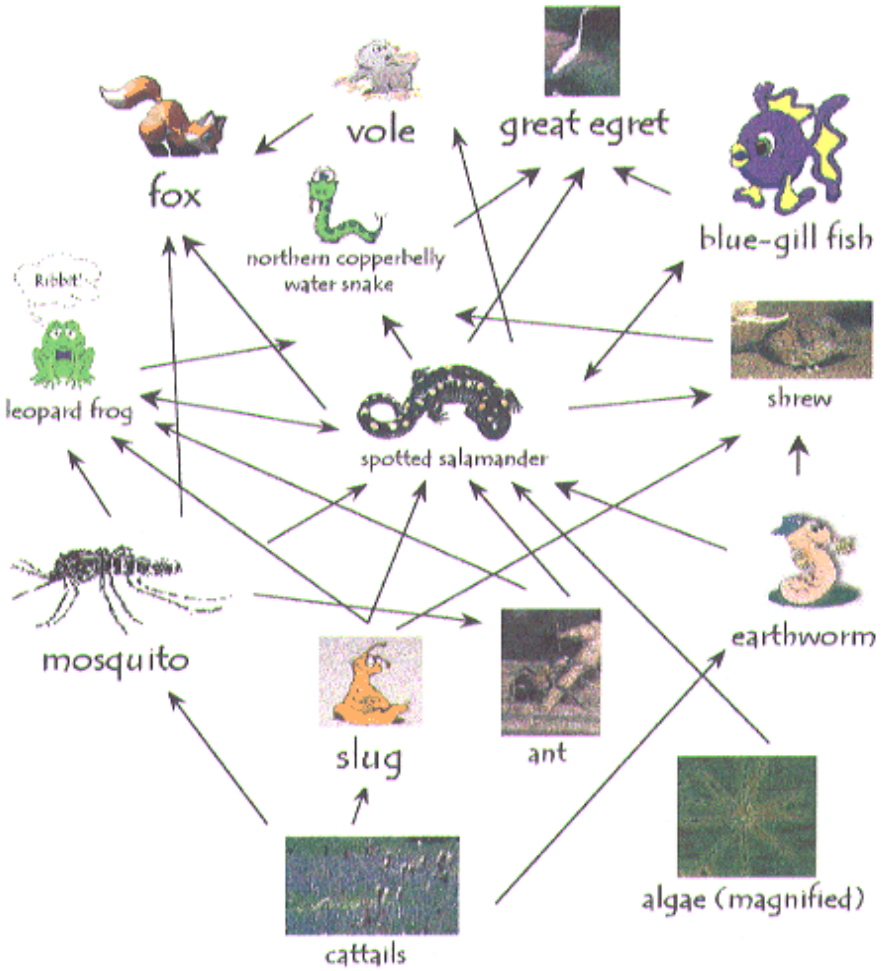
# Application: Ecological Food Web

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

## Food web graph

Node = species

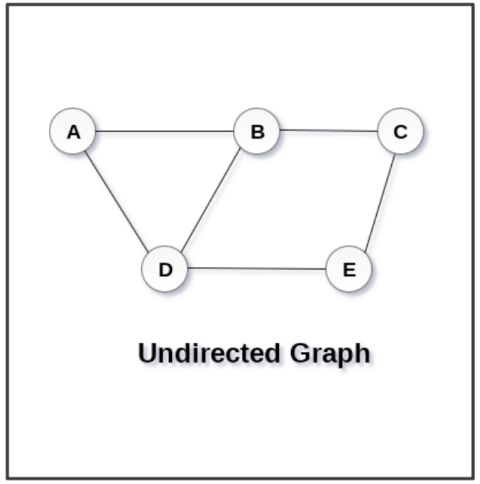
Edge = from prey to predator



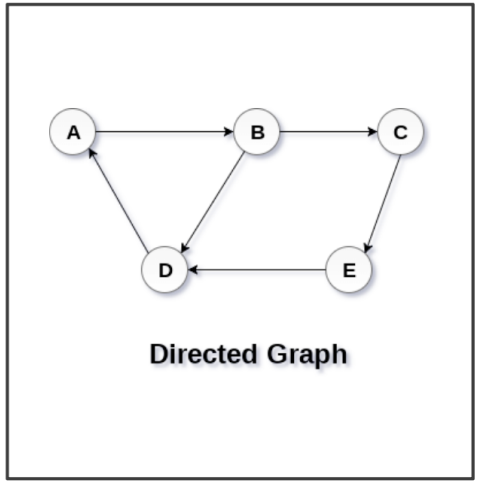
# More Applications

# Undirected V.S. Directed

- In an undirected graph, edges are not associated with the directions with them
  - If an edge exists between vertex A and B then the vertices can be traversed from B to A as well as A to B

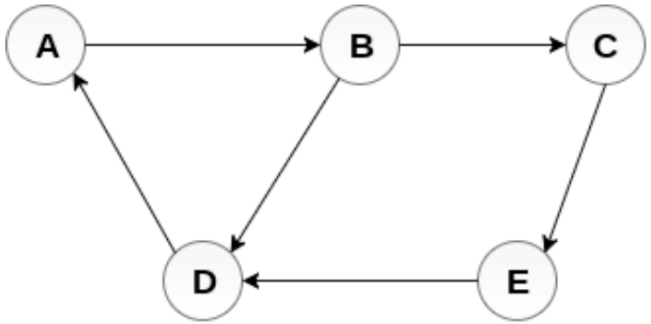



- In a directed graph, edges form an ordered pair
  - Edges represent a specific path from some vertex A to another vertex B
  - Node A is called initial node while node B is called terminal node



# Sequential Representation

- Directed: an entry $A_{ij}$ in the adjacency matrix will be 1 if there exists an edge directly from $v_i$ to $v_j$



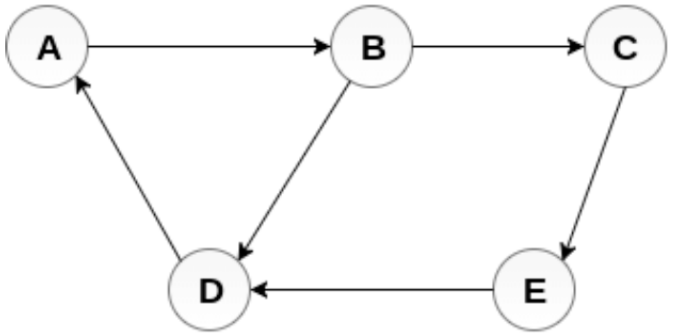

Directed Graph

| | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 | 0 |
| B | 0 | 0 | 1 | 1 | 0 |
| C | 0 | 0 | 0 | 0 | 1 |
| D | 1 | 0 | 0 | 0 | 0 |
| E | 0 | 0 | 0 | 1 | 0 |

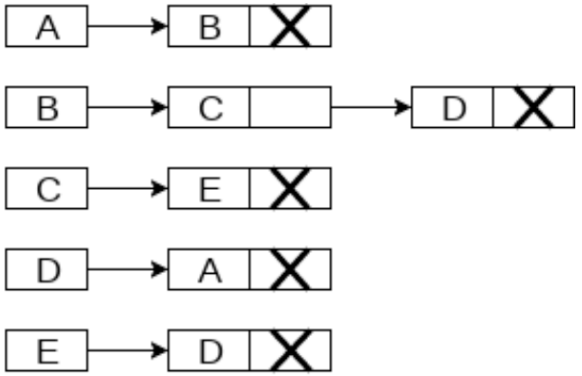
Adjacency Matrix

# Linked Representation

- Directed: The sum of the lengths of adjacency lists is equal to the number of edges




Directed Graph




Adjacency List

# Graph Search



- Directed reachability: Given a node $s$, find all nodes reachable from $s$
- Directed $s \rightsquigarrow t$ shortest path problem: Given two nodes $s$ and $t$, what is the length of a shortest path from $s$ to $t$?

coming soon!

- Graph Traversal: BFS and DFS extend naturally to directed graphs

# Strong Connectivity

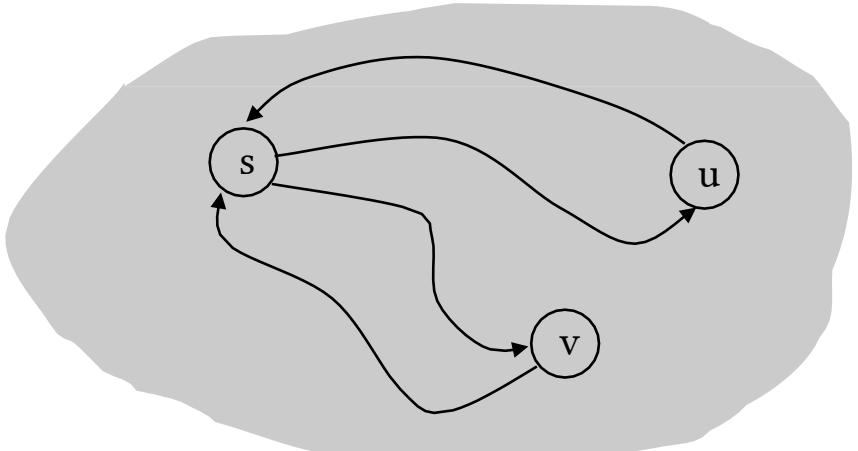
- Def. Nodes $u$ and $v$ are mutually reachable if there is both a path from $u$ to $v$ and also a path from $v$ to $u$
- Def. A graph is strongly connected if every pair of nodes is mutually reachable
- Lemma. Let $s$ be any node. $G$ is strongly connected iff every node is reachable from $s$, and $s$ is reachable from every node

Pf. $\Rightarrow$ Follows from definition

Pf. $\Leftarrow$ Path from $u$ to $v$: concatenate $u \rightsquigarrow s$ path with $s \rightsquigarrow v$ path

Path from $v$ to $u$: concatenate $v \rightsquigarrow s$ path with $s \rightsquigarrow u$ path ■

ok if paths overlap



# Strong Connectivity: Algorithm



- Theorem. Can determine if $G$ is strongly connected in $O(m + n)$ time

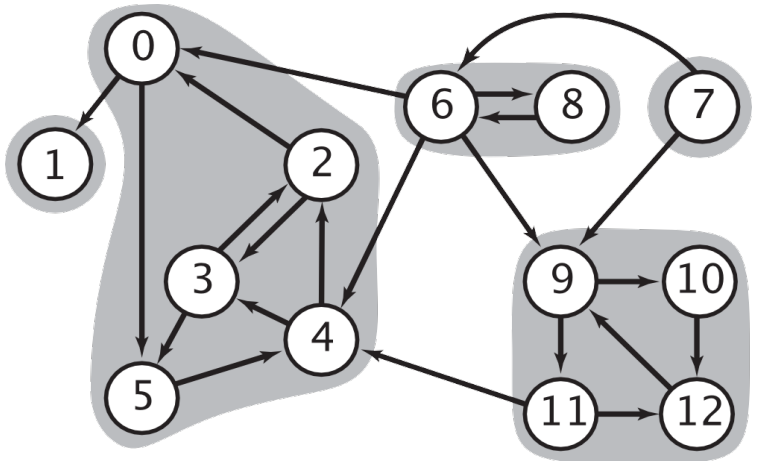
Pf.

- Pick any node $s$
- Run BFS from $s$ in $G$
- Run BFS from $s$ in $G$ reverse
- Return true iff all nodes reached in both BFS executions
- Correctness follows immediately from previous lemma ▪

reverse orientation of every edge in G

# Strong Components

- Def. A strong component is a maximal subset of mutually reachable nodes




Theorem. [Tarjan 1972] Can find all strong components in $O(m+n)$ time

SIAM J. COMPUT.
Vol. 1, No. 2, June 1972

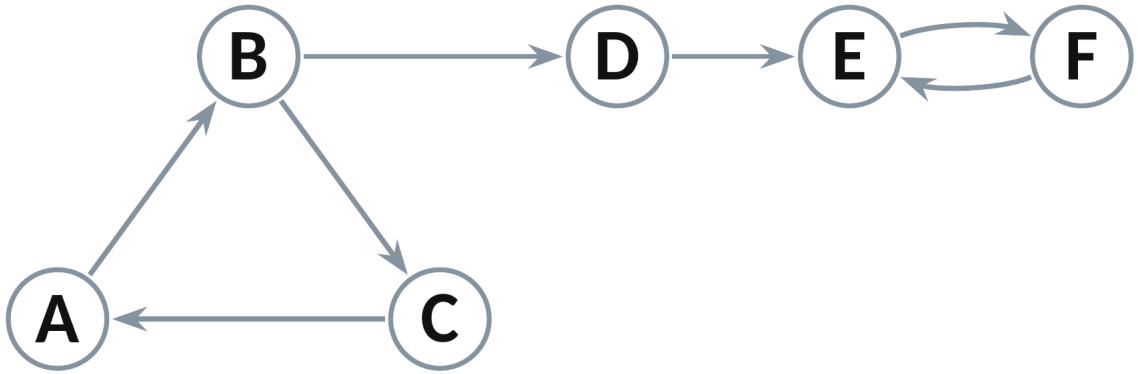
**DEPTH-FIRST SEARCH AND LINEAR GRAPH ALGORITHMS***

ROBERT TARJAN†

**Abstract.** The value of depth-first search or "backtracking" as a technique for solving problems is illustrated by two examples. An improved version of an algorithm for finding the strongly connected components of a directed graph and an algorithm for finding the biconnected components of an undirect graph are presented. The space and time requirements of both algorithms are bounded by $k_1V + k_2E + k_3$ for some constants $k_1$, $k_2$, and $k_3$, where $V$ is the number of vertices and $E$ is the number of edges of the graph being examined.

# Tarjan's Algorithm: A Complete Example

**Goal:** find all strongly connected components (SCCs).
Output each component as a set of vertices.




- Start DFS at $A$.
- At $B$, explore $B \to C$ before $B \to D$.

# First Visits and the SCC Stack

A vertex is **discovered** when DFS visits it **for the first time**.

On entering DFS($u$), before examining its outgoing edges:

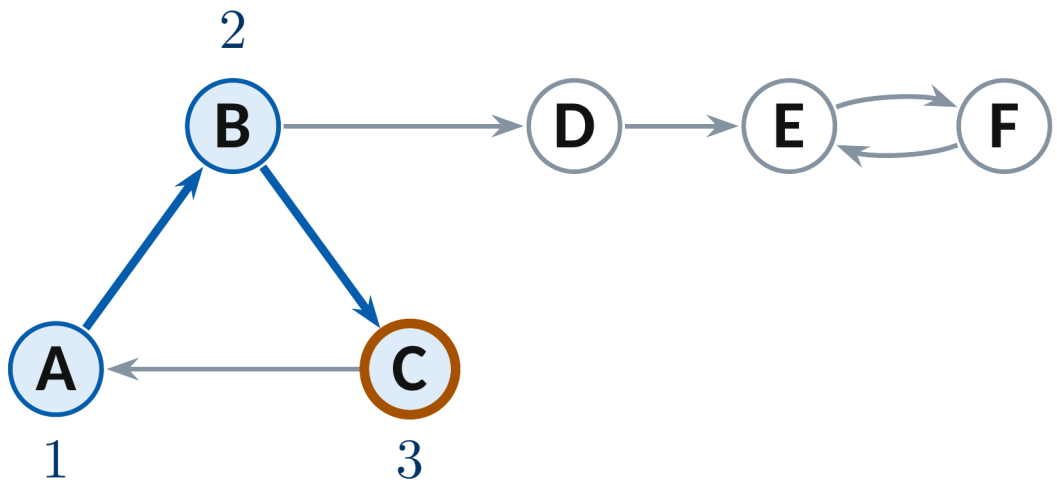
1. Set $num[u]$ to the next discovery number.
   The first vertex visited gets $1$, the second gets $2$, and so on.
2. Push $u$ onto the **stack** $S$.

Both actions happen **once per vertex**.

$S$ stores visited vertices that have not yet been reported as part of an SCC.

First visit to $A$: $num[A] = 1$ $S$: **A** top

# Example: First Visits to $A$, $B$, and $C$



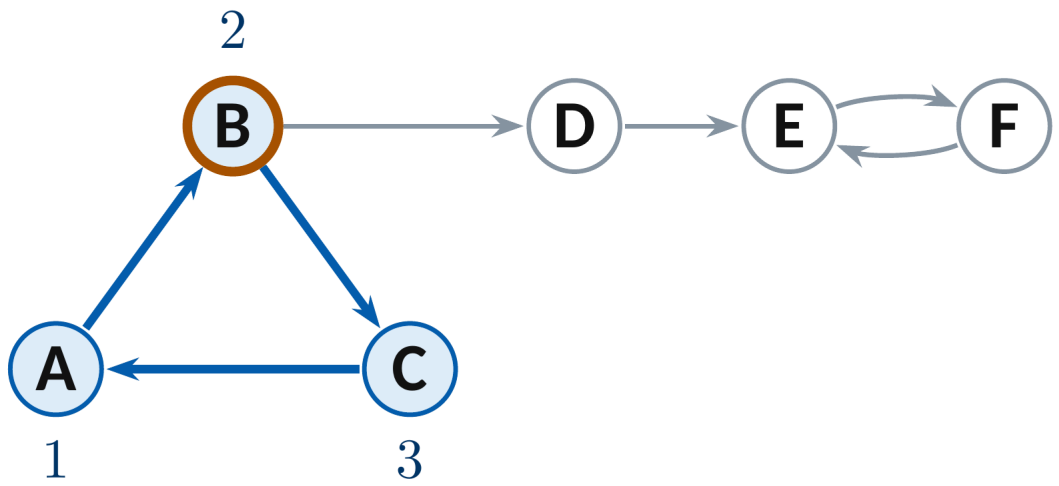

- DFS first visits $A$, then $B$, then $C$.
- Assign discovery numbers $1, 2, 3$, respectively.
- Push each vertex at its first visit.

Blue: on $S$.
Orange outline: current DFS call.

Visit $A$: | A | top  Visit $B$: | A | B | top  Visit $C$: | A | B | C | top

**SCCs reported so far: none.**

# Example: Returning from $C$



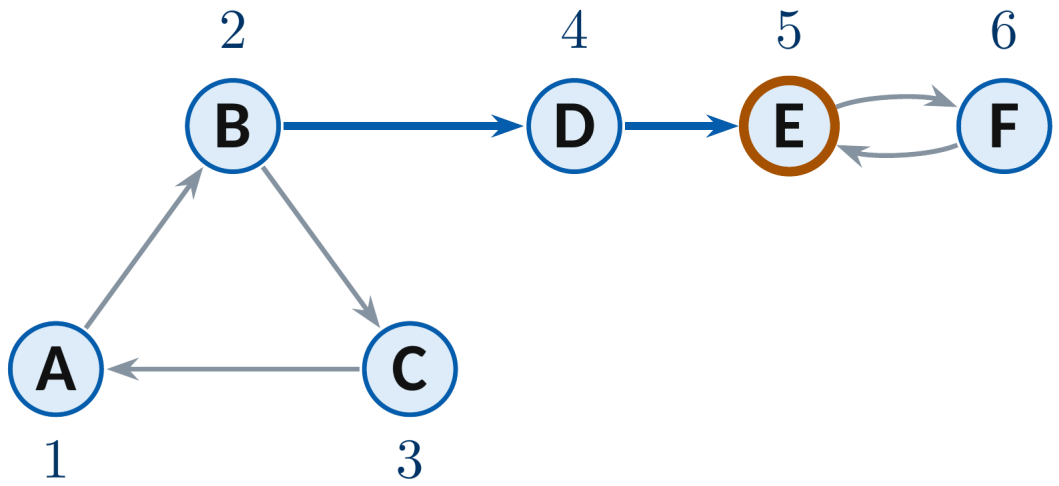

- $C \to A$: $A$ is already visited. No new call or push.
- $A, B, C$ lie on a cycle. Keep all three on $S$.
- DFS($C$) returns to $B$. The edge $B \to D$ is still unexplored.

DFS calls: | A | B | top

SCC stack: | A | B | C | top

**SCCs reported so far: none.**

# Example: Continuing to $D$, $E$, and $F$

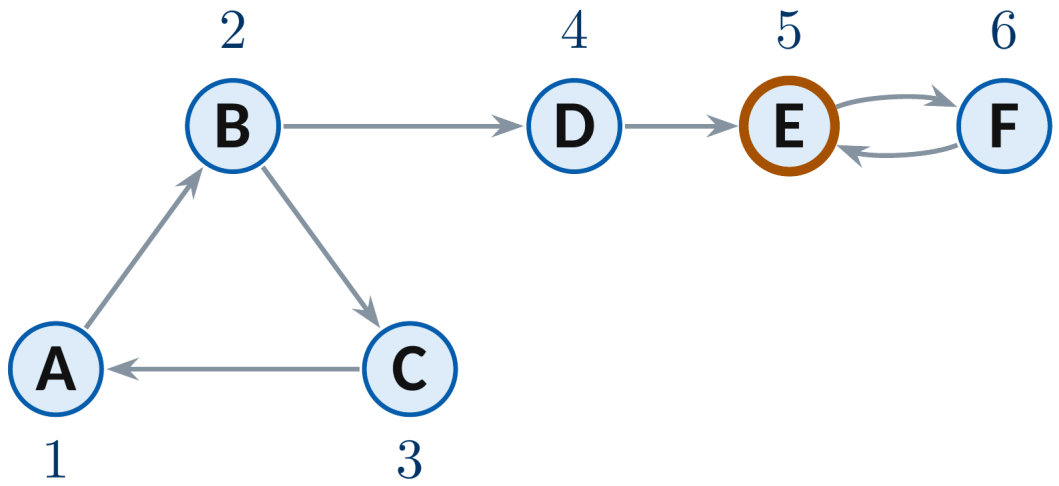



- DFS visits $D, E, F$, in that order. Number and push each vertex.
- $F \to E$ reaches a vertex already on $S$.
- DFS($F$) returns to $E$; $F$ stays on $S$.

$S$: | A | B | C | D | E | F | top

**SCCs reported so far: none.**

# Example: Reporting the First SCC




- All outgoing edges of $E$ and $F$ have been examined.
- $E$ and $F$ reach each other. Neither can reach $A, B, C$, or $D$.
- At the end of DFS($E$), pop $F$, then $E$.

Before:

| A | B | C | D | E | F |
|---|---|---|---|---|---|

top

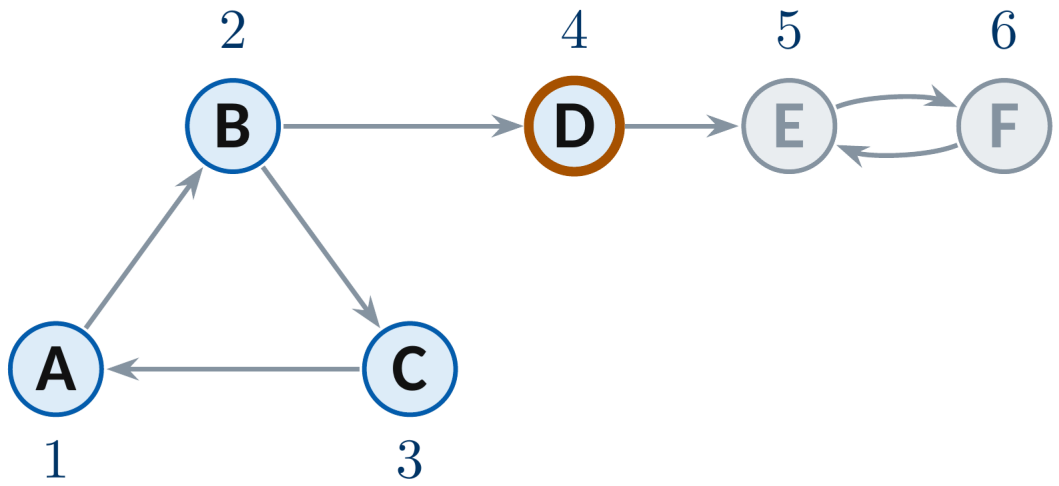
After:

| A | B | C | D |
|---|---|---|---|

top

**First SCC reported: $\{E, F\}$. Then return to $D$.**

# Example: Reporting the Second SCC




Gray: already reported as part of an SCC.

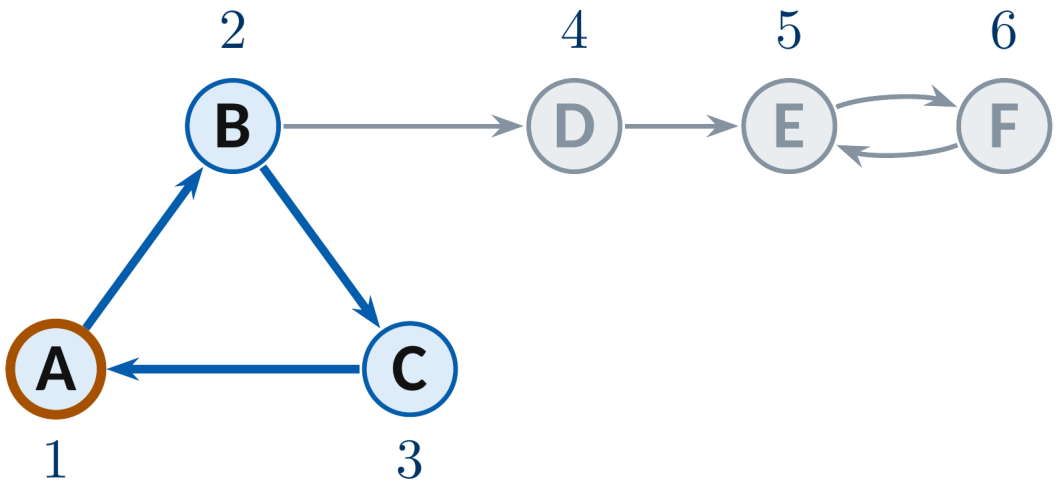
- DFS($E$) has returned to $D$.
- $D$ reaches $E$ and $F$, but neither can reach $D$.
- At the end of DFS($D$), pop $D$ and report $\{D\}$.
- Return to $B$.

Before: | A | B | C | D | top

After: | A | B | C | top

**SCCs reported so far: $\{E, F\}$, then $\{D\}$.**

# Example: Reporting the Last SCC




- DFS($B$) returns to $A$; keep $B$ on $S$.
- $A, B, C$ reach each other. All edges from these vertices have been examined.
- At the end of DFS($A$), pop $C, B, A$ together.

Before: | A | B | C | top $\longrightarrow$ $S = \varnothing$

**SCCs reported, in order:** $\{E, F\}, \{D\}, \{A, B, C\}$.

# The Rule for Reporting an SCC

Let $r$ be the **first vertex visited** in an SCC.
We report that SCC **at the end of DFS($r$)**.

| SCC | First vertex $r$ | When reported |
|---|---|---|
| $\{E, F\}$ | $E$ (number $5$) | End of DFS($E$) |
| $\{D\}$ | $D$ (number $4$) | End of DFS($D$) |
| $\{A, B, C\}$ | $A$ (number $1$) | End of DFS($A$) |

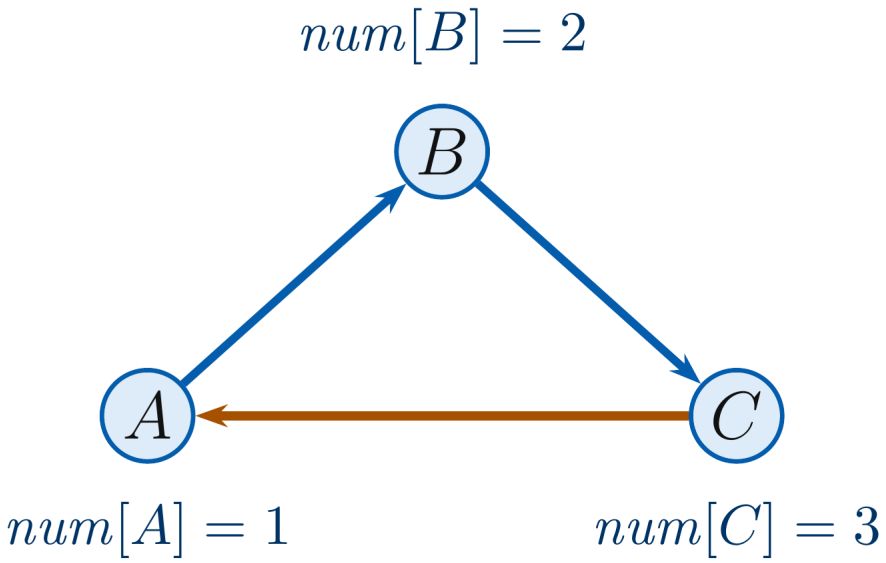
At the end of DFS($u$), determine whether $u$ was visited first in its SCC.

**For this test, we maintain an additional value $low[u]$.**

- A value smaller than $num[u]$ tells us that $u$ can reach an earlier-visited vertex in its SCC.

# Interpreting the Low-Link Value

We maintain $low[u]$ during DFS, starting with $low[u] = num[u]$.




1. **First, at $C$:**
   $C \to A$, and $A$ is still on $S$.
   $low[C] = num[A] = 1 < num[C]$.

2. **After DFS($C$) returns to $B$:**
   $B$ can reach $A$ through $C$.
   This gives $low[B] = 1 < num[B]$.

**If $low[u] < num[u]$, $u$ can reach an earlier vertex in the same SCC.**
**So $u$ is not the first vertex visited; keep $u$ on $S$.**

# Two Ways to Update Low-Link

Start with $low[u] = num[u]$. During DFS, keep the smallest value found.

## 1. Edge to a vertex already on $S$

$C$ examines $C \to A$.
Use $A$'s **discovery number**.

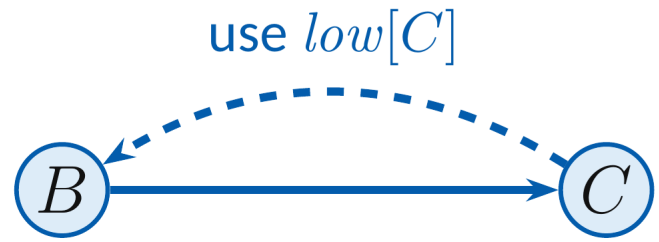



$low[C] = 3$ $num[A] = 1$

$low[C] \leftarrow \min(3, 1) = \mathbf{1}$.

## 2. Return from a recursive call

DFS($C$) returns to $B$.
Use $C$'s **low-link value**.




$low[B] = 2$ $low[C] = 1$

$low[B] \leftarrow \min(2, 1) = \mathbf{1}$.

**Ignore edges to already reported SCCs.**

# Computing the Low-Link Value



Initially, $low[u] = num[u]$. For each outgoing edge $u \to v$:

| Status of $v$ | Action |
|---|---|
| Unvisited | Call DFS($v$), then update: $low[u] \leftarrow \min(low[u], low[v])$. |
| Already visited and on $S$ | Use the discovery number of $v$: $low[u] \leftarrow \min(low[u], num[v])$. |
| Already reported in an SCC | Do not change $low[u]$. |

# The Component Test

Apply the test **at the end of DFS($u$)**, after all outgoing edges of $u$ have been examined.

- If $low[u] < num[u]$, $u$ can reach an earlier vertex in the same SCC. $u$ is not the first vertex visited. Keep $u$ on $S$.
- If $low[u] = num[u]$, $u$ is the first vertex visited in that SCC. Pop from $S$ until $u$ is popped; output these vertices as one SCC.




**At the end of DFS($E$), $low[E] = num[E] = 5$. Report $\{E, F\}$.**

# Applying the Test to the Running Example



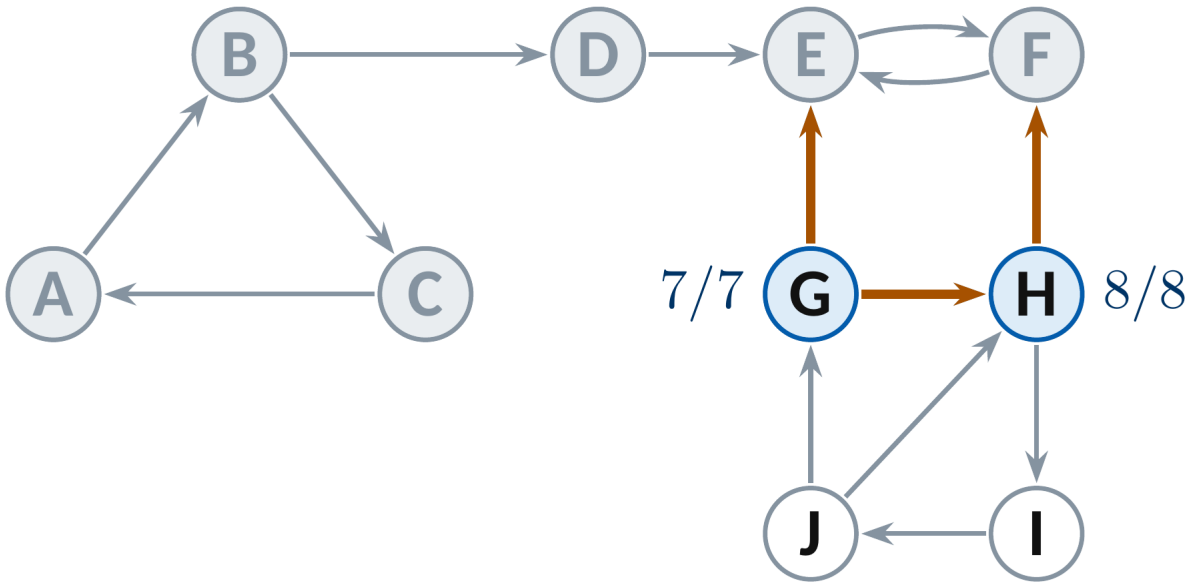
At the end of DFS($u$), compare $low[u]$ with $num[u]$.

| Call ending | $num[u]$ | $low[u]$ | Action on $S$ |
|---|---|---|---|
| DFS($C$) | 3 | 1 | Keep $C$ on $S$ |
| DFS($F$) | 6 | 5 | Keep $F$ on $S$ |
| DFS($E$) | 5 | 5 | Pop $F, E$; report $\{E, F\}$ |
| DFS($D$) | 4 | 4 | Pop $D$; report $\{D\}$ |
| DFS($B$) | 2 | 1 | Keep $B$ on $S$ |
| DFS($A$) | 1 | 1 | Pop $C, B, A$; report $\{A, B, C\}$ |

$C \to A$ gives $low[C] = 1$; returning to $B$ gives $low[B] = 1$.
After DFS($E$) returns: $low[D] = \min(4, 5) = 4$.

# Edges to a Completed SCC

DFS($A$) has finished; $S$ is empty.
$G$ is still unvisited, so start DFS($G$).

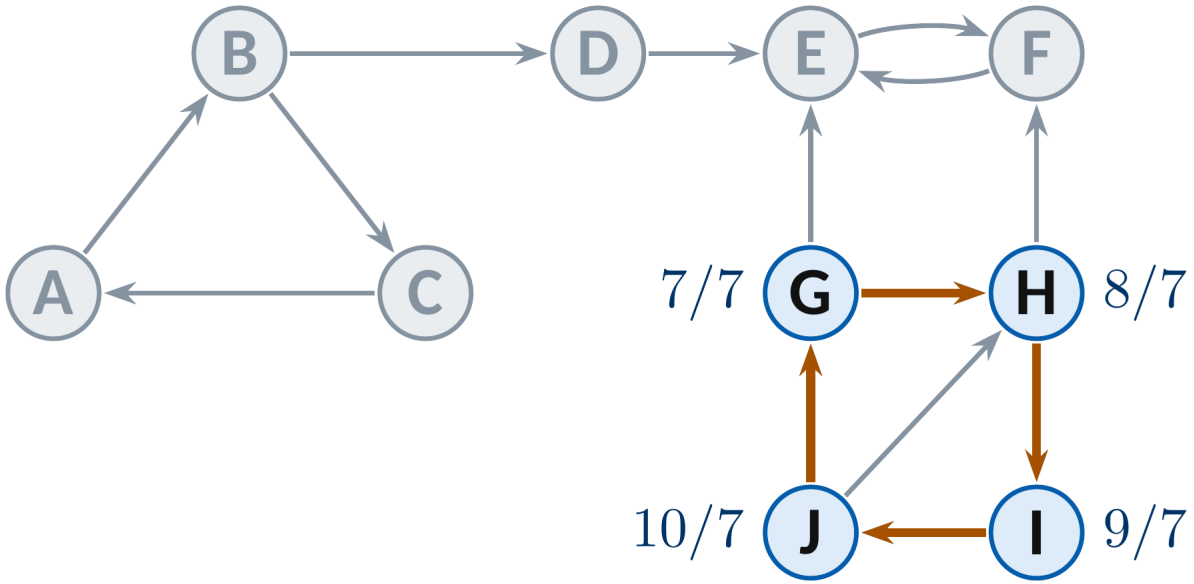



Vertex labels: $num/low$.

- $G \to E$ reaches a completed SCC.
  Keep $low[G] = 7$.
- Next, visit $H$.
  $H \to F$ also reaches a completed SCC.
  Keep $low[H] = 8$.

**Ignore edges to vertices already removed from $S$.**

# Example: Completing $\{G, H, I, J\}$




- $J \to G$ gives $low[J] = 7$.
- Returning to $I$ and then $H$ gives $low[I] = low[H] = 7$.
- At $G$, $low[G] = num[G] = 7$.
- Pop $J, I, H, G$ together.

**Final SCCs:** $\{A, B, C\}$, $\{D\}$, $\{E, F\}$, $\{G, H, I, J\}$.

# Tarjan's Algorithm: Setup and Outer Loop



### Initialize

- Every $num[u]$ is undefined.
- $S$ is empty; $nextNumber = 1$.

### Run DFS

**for** each vertex $u$ **do**
  **if** $num[u]$ is undefined **then**
    DFS($u$)

### Track membership in $S$

$onStack[u]$ is true exactly when $u$ is on $S$.

- Initially: all flags are false.
- Push $u$: set its flag to true.
- Pop $u$: set its flag to false.

**The outer loop starts DFS from each vertex that is still unvisited.**

# Tarjan's Algorithm: DFS



**procedure** DFS($u$)

$num[u] \leftarrow nextNumber$; $nextNumber \leftarrow nextNumber + 1$

$low[u] \leftarrow num[u]$

Push $u$ onto $S$

**for** each outgoing edge $u \rightarrow v$ **do**

**if** $num[v]$ is undefined **then**

DFS($v$)

$low[u] \leftarrow \min(low[u], low[v])$

**else if** $v$ is on $S$ **then**

$low[u] \leftarrow \min(low[u], num[v])$

**if** $\boldsymbol{low[u] = num[u]}$ **then**

Pop vertices from $S$ until $u$ is popped, including $u$

Report the popped vertices together as one SCC

# Running Time of Tarjan's Algorithm



Use an adjacency list.

- Each vertex is discovered once.
- Each directed edge is examined once.
- Each vertex is pushed and popped once.
- Each number update and stack-membership test takes $O(1)$ time.

Total time: $O(|V| + |E|)$ Extra space: $O(|V|)$

**Let $r$ be the first vertex visited in an SCC. Report that SCC at the end of DFS($r$).**

# DAG & Topological Ordering

# Directed Acyclic Graphs

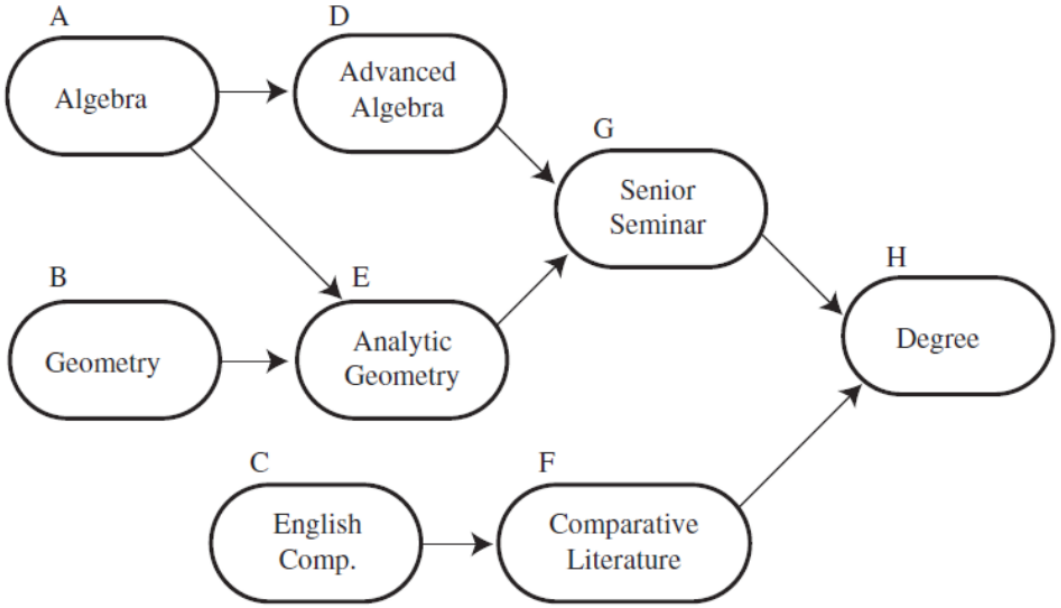


- Def. A DAG is a directed graph that contains no directed cycles

- Def. A topological order of a directed graph $G = (V, E)$ is an ordering of its nodes as $v_1, v_2, \ldots, v_n$ so that for every edge $(v_i, v_j)$ we have $i < j$

# Precedence Constraints

- Precedence constraints. Edge $(v_i, v_j)$ means task $v_i$ must occur before $v_j$
- Applications
  - Course prerequisite graph: course $v_i$ must be taken before $v_j$
  - Compilation: module $v_i$ must be compiled before $v_j$
  - Pipeline of computing jobs: output of job $v_i$ needed to determine input of job $v_j$



# Directed Acyclic Graphs



Lemma. If $G$ has a topological order, then $G$ is a DAG

Pf. [by contradiction]

- Suppose that $G$ has a topological order $v_1, v_2, \ldots, v_n$ and that $G$ also has a directed cycle $C$
- Let $v_i$ be the lowest-indexed node in $C$, and let $v_j$ be the node just before $v_i$; thus $(v_j, v_i)$ is an edge
- By our choice of $i$, we have $i < j$
- On the other hand, since $(v_j, v_i)$ is an edge and $v_1, v_2, \ldots, v_n$ is a topological order, we must have $j < i$, a contradiction ▪

# Directed Acyclic Graphs



Lemma. If $G$ has a topological order, then $G$ is a DAG

Q. Does every DAG have a topological ordering?

Q. If so, how do we compute one?

# Directed Acyclic Graphs



Lemma. If $G$ is a DAG, then $G$ has a node with no entering edges

Pf. [by contradiction]

- Suppose that $G$ is a DAG and every node has at least one entering edge
- Pick any node $v$, and begin following edges backward from $v$. Since $v$ has at least one entering edge $(u, v)$ we can walk backward to $u$
- Then, since $u$ has at least one entering edge $(x, u)$, we can walk backward to $x$
- Repeat until we visit a node, say $w$, twice
- Let $C$ denote the sequence of nodes encountered between successive visits to $w$. $C$ is a cycle ▪

# Directed Acyclic Graphs



Lemma. If $G$ is a DAG, then $G$ has a topological ordering

Pf. [by induction on $n$]

- Base case: true if $n = 1$
- Given DAG on $n > 1$ nodes, find a node $v$ with no entering edges
- $G - \{v\}$ is a DAG, since deleting $v$ cannot create cycles
- By inductive hypothesis, $G - \{v\}$ has a topological ordering
- Place $v$ first in topological ordering; then append nodes of $G - \{v\}$ in topological order. This is valid since $v$ has no entering edges ▪

---

To compute a topological ordering of $G$:

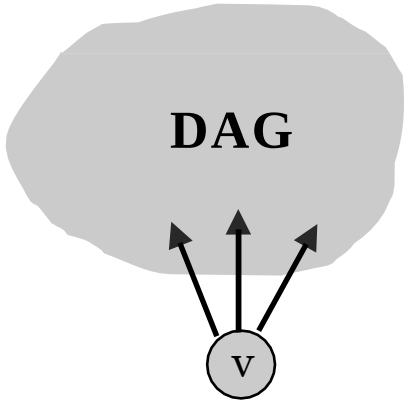
Find a node $v$ with no incoming edges and order it first

Delete $v$ from $G$

Recursively compute a topological ordering of $G-\{v\}$

  and append this order after $v$

---



# Topological Sorting Algorithm



- Theorem. Algorithm finds a topological order in $O(m + n)$ time
- Pf.
  - Maintain the following information:
    - *count*($w$) = remaining number of incoming edges
    - $S$ = set of remaining nodes with no incoming edges
  - Initialization: $O(m + n)$ via single scan through graph
  - Update: to delete $v$
    - *remove* $v$ *from* $S$
    - *decrease count(w) for all edges from* $v$ *to* $w$*; and add* $w$ *to* $S$ *if count(w) hits* 0
    - *this is O(1) per edge* ▪

- Topological-sort cannot handle graphs with cycles!

# Summary



- Graphs definition
- Graphs representation
- Graph search algorithms
- Connected components in directed/undirected graphs
- Tarjan's Algorithm
- DAGs and Topological orders